Plain-English Companion to "Fourier Neural Operators for 2D Navier–Stokes"

A guide for our own understanding
COMP 395 — Deep Learning

May 2026

What this document is. The formal report uses technical language for the professor. This companion explains everything in plain English so *we* can follow along. Read them side by side, or read this first.

What We Did — In One Paragraph

We trained two AI models to watch a swirling fluid (think smoke rings or water spinning in a sink) and predict what it will look like one moment later. The first model, called FNO, was designed specifically with fluid physics in mind. The second, called U-Net, is a general-purpose image model we used as a comparison point.

FNO won on every single metric: it was three times more accurate, ran more than twice as fast, and used six times fewer "brain cells" (parameters). On top of that, FNO could handle pictures of the fluid at resolutions it had *never seen during training* and still did a great job — U-Net fell apart completely at those new resolutions.

The big takeaway: when you design an AI model that understands the *shape* of the problem it is solving (fluid physics), it beats a generic model even if that generic model is much larger.

1 Introduction — Why Does Any of This Matter?

1.1 Simulating Fluids Is Expensive

Fluids (liquids and gases) follow rules that scientists have known for over 150 years — the Navier–Stokes equations. These equations are correct and precise, but actually *solving* them on a computer is brutally slow.

Analogy. Imagine a weather forecast that requires a supercomputer running for six hours just to tell you what tomorrow looks like. That is roughly the situation with high-fidelity fluid simulation. The equations are not wrong; they are just expensive.

To predict the weather for the next 24 hours at high resolution, scientists divide the atmosphere into millions of tiny boxes, apply the equations to each box, and step forward in time — millions of times. This works, but it takes enormous computing resources.

1.2 What We Want Instead

Instead of solving the equations from scratch every time, we want to *train* a neural network to look at a snapshot of the fluid and immediately predict the next snapshot — skipping all the expensive arithmetic.

This is called a **surrogate model**: it *stands in* for the real simulation. If it works, a single forward pass through the neural network (milliseconds) replaces the expensive numerical solver (minutes to hours).

1.3 The Catch

The tricky part is that the fluid is represented as a *grid of values* (a 2D image, basically). If you train a standard image neural network at one image resolution, it generally cannot handle a different resolution without retraining. That is a big problem if you want to use the same model at different scales.

FNO was designed to solve this problem by working in *frequency space* instead of pixel space. We will explain exactly what that means in Section 2.

2 Background — Building Blocks

2.1 What Is a Vorticity Field?

We do not directly track how fast every tiny piece of fluid is moving. Instead, we track **vorticity** — a number at every point in the fluid that describes how much the fluid is *spinning* there.

> **Analogy.** Picture a river. Where the water flows straight, vorticity is near zero. Where it curls into a whirlpool, vorticity is large. Positive vorticity = spinning clockwise; negative vorticity = spinning counter-clockwise.

In our case, the fluid lives on a square grid (128×128 points). Each grid point holds one vorticity number. So one "snapshot" of the fluid is just a 128×128 grid of numbers — essentially a grayscale image where brightness = vorticity.

2.2 What Are the Navier–Stokes Equations?

The Navier–Stokes equations describe how fluid velocity changes over time. In plain terms, the equation says:

"How the fluid's motion changes right now"
= *"How it is pushing itself around"*
+ *"How pressure is pushing it"*
+ *"How viscosity (thickness) slows it down"*
+ *"Any external force acting on it"*

The formal version looks like this:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f}$$

Each symbol maps directly onto the plain-English list above. $\mathbf{u}$ is velocity, p is pressure, ν is viscosity (how thick the fluid is — honey has high ν, air has low ν), and $\mathbf{f}$ is external force.

The reason this is hard is the middle term on the left, $(\mathbf{u} \cdot \nabla)\mathbf{u}$: the fluid's own motion affects its future motion (it is *nonlinear*). That self-interaction is what causes chaotic, turbulent behaviour.

2.3 Kolmogorov Forcing — Why the Fluid Swirls

Our dataset uses a specific external force called **Kolmogorov forcing**: a sinusoidal push that continuously pumps energy into the fluid at a large spatial scale (imagine someone slowly waving a giant paddle back and forth). At the viscosity we use ($\nu = 10^{-4}$, i.e., very thin fluid), that energy cascades down to smaller and smaller scales, creating complex swirling patterns — 2D turbulence.

The fluid in our dataset is not a real physical substance like water or air; it is a dimensionless benchmark designed to have the right level of complexity for testing neural networks.

2.4 What Is a Neural Operator?

A standard neural network takes in a fixed-length list of numbers and outputs another fixed-length list. If you train it on 128×128 images, you cannot feed it a 256×256 image.

A **neural operator** is different: it is designed to map one *function* to another function, regardless of how finely that function is sampled.

> **Analogy.** A regular neural network is like a calculator that only works on numbers between 0 and 100. A neural operator is like a formula: it works for any input, no matter how you choose to measure it.

For fluids, this means: train on 128×128, then apply to 256×256 — no retraining needed. FNO is one way to build a neural operator.

2.5 How FNO Works — Step by Step

FNO's key insight is that fluid dynamics is *smooth at large scales*: the big swirling patterns (low frequencies) carry most of the physically important information. FNO exploits this by doing its main computation in frequency space.

Step 1 — FFT. Take the 128×128 vorticity grid and apply a 2D Fast Fourier Transform (FFT). This converts the image from "how much vorticity at each grid point" into "how much of each swirling pattern is present." Low frequencies = large, smooth swirls; high frequencies = fine, rapid wiggles.

Step 2 — Truncate. Throw away everything above frequency mode 12 in each direction. We keep only the 12 lowest-frequency components — the large-scale structure. This dramatically reduces the amount of information to process and acts as a physics-motivated compression.

Step 3 — Multiply by learned weights. Each kept frequency mode is multiplied by a learned complex number. These complex weights are what the training process optimises. Conceptually, this is the model learning "how much to amplify or dampen each frequency when stepping forward in time."

Step 4 — Inverse FFT. Convert back from frequency space to grid space. The result is a transformed vorticity field at the original resolution.

Step 5 — Add a local correction. A standard 1×1 convolution is added in parallel to the Fourier step. This handles anything that is very local (fine-grained effects that the truncated frequencies missed).

Why this is resolution-independent. The FFT produces frequency components; their physical meaning does not change when you use a finer or coarser grid. So the learned weights remain valid at any resolution. That is the core reason FNO can do zero-shot super-resolution.

2.6 How U-Net Works

U-Net is a classic convolutional encoder–decoder. Think of it as a "zoom out, transform, zoom back in" pipeline:

1. **Encoder**: repeatedly shrink the image by half while doubling the number of feature channels. Four times = down to 8×8 with 512 channels.
2. **Bottleneck**: the most compressed representation of the input.
3. **Decoder**: upsample back to the original size, step by step. At each step, paste in the matching encoder features (skip connections) to recover fine detail.

U-Net is extremely good at image-to-image tasks and is the de facto standard in medical image segmentation. The limitation: all operations use fixed-size 3×3 kernels that look at a specific physical neighbourhood. When the resolution changes, those kernels cover a different physical area, which is why U-Net does not generalise across resolutions.

3 Dataset — What We Actually Fed the Models

3.1 Where the Data Comes From

We used a publicly available dataset from the neuraloperator project, downloaded from Zenodo (a data-sharing platform for scientists). File: `nsforcing_128.tgz`, about 1.5 GB.

3.2 What One Training Example Looks Like

Each example is a pair of two 128×128 grids:

- **Input** x: the vorticity field right now.
- **Target** y: the vorticity field one timestep later.

The model's job: look at x, predict y.

3.3 What the Data Looks Like

The input fields are calm: numbers mostly between -0.76 and 0.82. The target fields are more energetic: numbers between -3.49 and 3.93. This makes sense physically — the Kolmogorov forcing is continuously pumping energy in, so the fluid is getting more turbulent over time.

> **Key point.** The targets are roughly 20× more variable than the inputs. This means the model cannot just copy the input and call it done — it needs to actually predict the physics.

We have 10,000 training pairs and 2,000 test pairs.

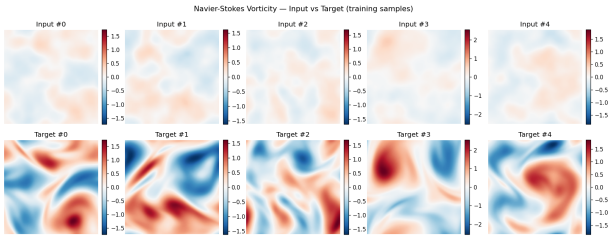

Figure 1: **What the data looks like.** Five examples from the training set. **Top row:** the input vorticity field the model sees. **Bottom row:** the target field it must predict, one timestep later. Red = fluid spinning clockwise; blue = fluid spinning counter-clockwise; white = not spinning. Notice the bottom row is much more dramatic — more red and blue, bigger patches — because the fluid has become more turbulent.

4 Methods — How We Trained Everything

4.1 The Loss Function

During training, after the model makes a prediction $\hat{u}$, we need to measure how wrong it was compared to the true answer u. We use **relative L^2 error**:

$$\text{error} = \frac{\|\hat{u} - u\|_2}{\|u\|_2}$$

In plain English: compute the size of the mistake, then divide by the size of the true answer. This gives a percentage-like score:

- 1.0 = your prediction is as wrong as a random guess (terrible).
- 0.1 = your mistake is 10% the size of the true answer (decent).
- 0.03 = your mistake is only 3% the size (great).

4.2 Training Details

Both models were trained for 100 epochs (100 full passes over the training set). We used the Adam optimiser — a standard recipe for updating the model's weights to reduce error — starting with a learning rate of 10^{-3} that slowly decays to 10^{-5} over training (cosine schedule).

> **Analogy — learning rate.** The learning rate controls how big a step you take when adjusting the model after each mistake. Too large: you overshoot and never settle. Too small: you take forever. Cosine scheduling starts large (fast learning) and gradually slows down (fine-tuning), like how you take big strides to cross a room and then small careful steps when threading a needle.

Hardware: an NVIDIA RTX 2060 GPU (6 GB video memory), which is a mid-range gaming card — not a supercomputer. Both models trained overnight.

4.3 What "Parameters" Are

A neural network is basically a huge collection of numbers (called **parameters** or **weights**). Training adjusts these numbers until the network makes good predictions. More parameters = more capacity to learn complex patterns, but also more memory and slower inference.

- FNO: **4.7 million** parameters
- U-Net: **31 million** parameters (6.6× more)

5 Results — What We Found

5.1 Main Results: Accuracy and Speed

Table 1 summarises the head-to-head comparison.

Table 1: Head-to-head comparison on the test set. Lower rel-L^2 = more accurate. Higher throughput = faster.

	FNO	**U-Net**
Parameters	4.7M	31M
Rel-L^2 error	**0.035**	0.103
Throughput (samp/s)	**499**	227
Latency (ms/samp)	**2.0**	4.4

In plain English:

- **FNO's error is 3.5%** of the target's magnitude. U-Net's is 10.3%. FNO is about 3× more accurate.
- FNO processes **499 test samples per second**; U-Net manages 227. FNO is 2.2× faster.
- Each FNO prediction takes **2 milliseconds**. U-Net takes 4.4 ms.
- FNO achieves all of this with **6.6× fewer parameters**.

Why does FNO win on speed *and* accuracy? FNO's Fourier layers are implemented using the FFT algorithm, which is extremely efficient. U-Net's convolutions operate at full 128 × 128 resolution at every level. FNO's computation is concentrated on the 12 most important frequencies — most of the work happens in a much smaller space.

5.2 Looking at the Predictions

Figure 2 shows six test examples from worst to best FNO performance (the number below each column is the relative L^2 score for that sample).

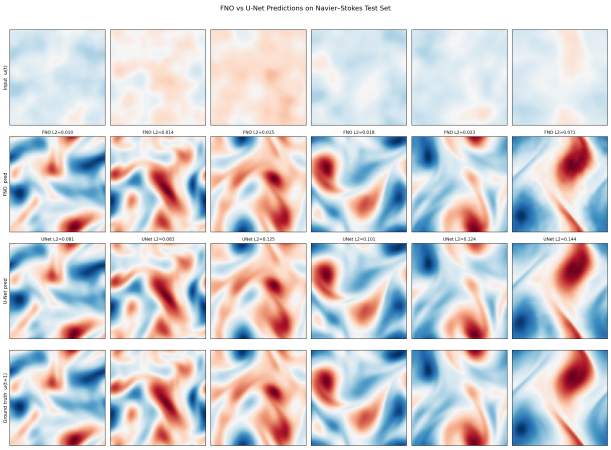

Figure 2: **Side-by-side predictions on six test samples. Row 1 (Input):** what the model is given — the current vorticity field. **Row 2 (FNO):** FNO's prediction of the next timestep. **Row 3 (U-Net):** U-Net's prediction. **Row 4 (Truth):** what actually happens next. FNO's predictions look almost identical to the ground truth. U-Net captures the rough shape but blurs out fine details and gets the magnitudes slightly wrong (colours are less intense).

5.3 Where the Errors Are

Figure 3 shows *where* each model makes mistakes, not just how much. Bright = big error; dark = small error.

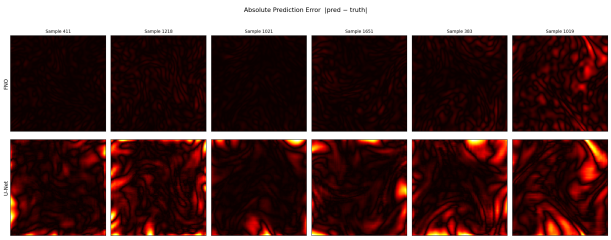

Figure 3: **Error maps: where each model goes wrong. Top row (FNO):** errors are small and spread evenly. **Bottom row (U-Net):** errors are large and concentrated at the edges of vortices — exactly the hardest parts, where the physics changes most rapidly. This tells us U-Net struggles most where the turbulence is most intense.

5.4 How Consistent Are the Models?

Figure 4 shows the distribution of errors across all 2,000 test samples — like a grade distribution for each model.

Figure 4: **Error distribution across the full test set.** Each bar counts how many test samples fell in that error range. **Blue (FNO):** most samples clustered tightly near 0.035. FNO is consistently good. **Red (U-Net):** spread much wider, with many samples having much higher error. Not only is U-Net less accurate on average, it is also less *reliable* — some samples it handles fine, others it gets badly wrong.

5.5 Zero-Shot Super-Resolution — The Coolest Result

What "zero-shot super-resolution" means. Both models were trained *only* on 128×128 images. We then asked them to make predictions at completely different resolutions — 64×64 (half the size) and 256×256 (double the size) — **without any additional training**. This is called "zero-shot" because the model gets zero examples at the new resolution.

Table 2 shows the results.

Table 2: Zero-shot super-resolution: errors at unseen resolutions. The model only trained on the 128×128 row.

Resolution	FNO error	U-Net error
64×64	**0.019**	0.476
128×128 (trained on)	**0.019**	0.103
256×256	**0.019**	0.594

FNO's error is **literally identical** across all three resolutions (0.019 each time, rounded to three decimal places). It does not care how many pixels you give it — it is solving the same underlying physics problem.

U-Net's error at 256×256 is 0.594 — worse than a coin flip (recall, 1.0 = random guess). Its 64×64 error is 0.476. At both unseen resolutions, U-Net essentially fails.

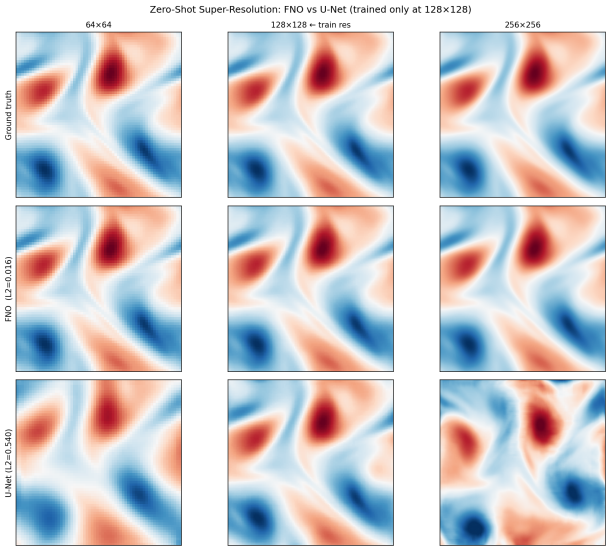

Figure 5: **Visual super-resolution comparison on one test sample. Columns:** 64×64, 128×128 (training res), 256×256. **Rows:** ground truth / FNO prediction / U-Net prediction. FNO looks sharp and accurate at every resolution. U-Net at 64×64 looks blurry and washed out. U-Net at 256×256 develops visible artefacts and the colours are wrong — it has no idea what to do at this resolution.

6 Discussion — Why Did This Happen?

6.1 Why Did FNO Beat U-Net So Convincingly?

The short answer: FNO's architecture *matches the structure of the problem.*

Fluid dynamics is global. A vortex forming on the left side of the domain immediately affects the pressure everywhere, including the right side. FNO's Fourier layers see the entire domain at once (via the FFT), so they can naturally model these global interactions.

U-Net's 3×3 convolutions only look at a tiny neighbourhood at a time. To propagate information from one side of a 128×128 field to the other, you need many successive layers. Even with 4 encoder levels, there is a limit to how much global context U-Net can capture.

> **Analogy.** Imagine predicting traffic jams. FNO is like having a helicopter view of the entire city at once. U-Net is like standing on the ground looking through a small window — you can see what is nearby, but you miss the big picture. Fluid dynamics needs the helicopter view.

6.2 Why Is FNO Resolution-Independent?

FNO works in frequency space. The Fourier Transform decomposes a signal into sine waves of different frequencies. Those frequencies have a fixed physical meaning (e.g., "a swirl that repeats 4 times across the domain") regardless of how finely the domain is sampled.

When you train FNO, it learns weights for "how much to amplify each frequency pattern." Those weights are valid at any resolution because the physical meaning of the frequencies does not change.

U-Net's 3×3 kernel, however, covers a *physical* neighbourhood of 3 pixels $\times$ (grid spacing). At 128×128 the grid spacing is $1/128$; at 256×256 it is $1/256$. The same kernel now covers half the physical area. Every single learned filter has a different physical meaning at the new resolution — which is why U-Net breaks.

6.3 Parameter Efficiency

FNO uses $6.6\times$ fewer parameters but wins. This is counterintuitive — more parameters usually means more capacity. But raw capacity is only useful if it is focused on the right things.

FNO's parameters are spent on learning the most important frequency components of the fluid. U-Net's 31 million parameters are spread across all the local pixel-to-pixel relationships, most of which are redundant or irrelevant for the physics.

6.4 What Are the Limitations?

- **Fixed viscosity.** Both models were trained at one viscosity value ($\nu = 10^{-4}$). We do not know if they generalise to thicker or thinner fluids.

- **Single-step only.** The model predicts one timestep at a time. If you wanted to simulate 100 timesteps into the future, you would need to feed the output back as input 100 times. Small errors would accumulate — we did not test this.

- **The super-resolution "ground truth" is approximate.** At 256×256, we do not have real simulation data. We created it by bilinearly upsampling the 128×128 data. This is a reasonable proxy but not perfect.

7 Conclusion — What Does It All Mean?

We showed that:

1. A neural network *can* learn to predict fluid dynamics accurately from data alone, without solving the equations numerically.

2. Designing the architecture to match the problem (Fourier layers for a Fourier-natured PDE) pays off enormously.

3. FNO is more accurate, faster, smaller, and more flexible than a standard CNN (U-Net) on this task.

4. The resolution-independence of FNO is not a lucky side effect — it is a direct consequence of how the architecture is built.

The broader lesson is about **inductive bias**: if you bake the right assumptions about your problem into your model's architecture, you can do more with less data and fewer parameters. This is especially powerful for physics problems where the underlying rules are known and can be used to guide design.

FNO is not a replacement for numerical simulation in all cases, but it is a compelling tool for cases where you need fast approximate predictions — weather nowcasting, real-time flow control, and interactive engineering design are all natural applications.

Glossary of Terms

Vorticity A measure of local rotation in a fluid. Positive = clockwise spin; negative = counter-clockwise spin.

Navier–Stokes equations The PDEs that govern all viscous fluid flow. Derived in the 19th century; still unsolved analytically in the general turbulent case.

PDE (Partial Differential Equation) An equation involving a function of multiple variables *and* its derivatives. Describes how something (temperature, velocity, vorticity) changes across both space and time simultaneously.

Viscosity (ν) How "thick" a fluid is. Honey has high viscosity; water has low viscosity; air has very low viscosity.

Reynolds number (Re) A dimensionless ratio of inertia to viscosity. High Re = turbulent. Our dataset: Re = 10,000 (very turbulent).

Neural operator A neural network that maps functions to functions, not fixed-length vectors. Resolution-independent by design.

FNO Fourier Neural Operator. Applies learned filters in frequency space (via FFT), enabling resolution-agnostic inference.

U-Net A CNN encoder–decoder architecture with skip connections. Originally designed for medical image segmentation.

FFT (Fast Fourier Transform) An efficient algorithm to convert a signal from the spatial domain to the frequency domain. Decomposes an image into sine waves of different frequencies.

Relative L^2 error $\|\text{prediction} - \text{truth}\|_2 / \|\text{truth}\|_2$. A scale-free error metric. Score of 1.0 = random guess; 0.0 = perfect.

Zero-shot super-resolution Evaluating a model at a resolution it has never seen during training, without any fine-tuning.

Parameters / weights The numbers inside a neural network that are adjusted during training. More parameters = more capacity (not always better).

Epoch One complete pass through the entire training dataset.

Learning rate How large a step the optimiser takes when updating parameters after each batch of training examples.

Cosine annealing A learning rate schedule that starts high and gradually decays in a smooth cosine curve to near zero.

Inductive bias Assumptions baked into a model's architecture about the structure of the problem. Good inductive bias = better performance with less data.